

Secure Software Development 1

Installed GitGuardian



GitGuardian - Secrets Security

GitGuardian | 3,364 | ★★★★★ (9)

A safety net to help developers safeguard their sensitive information (passwords, API k

[Disable](#) [Uninstall](#) ☒ Auto Update 

[DETAILS](#) [FEATURES](#) [CHANGELOG](#)

GitGuardian for VS Code

Detect secrets in real time and retroactively across extensive sources, with no limit to your team's scale. GitGuardian

Installed SonarQube



SonarQube for IDE

SonarSource Extension name [sonarsource.com](#) | 3,663,080 | ★★★★★ ☆ (117)

Advanced linter to detect & fix coding issues locally in JS/TS, Python, Java, C#, C/C

[Disable](#) [Uninstall](#) ☒ Auto Update 

[DETAILS](#) [FEATURES](#) [CHANGELOG](#)

SonarQube for IDE: Visual Studio Code (formerly SonarLint)

SonarQube for IDE by [Sonar](#) is a free, sophisticated static analysis tool that enhances your code quality and security.

Included .gitignore- used gitignore to prevent the essential secret data from getting uploaded on github repository (shown in the screenshot below)

Working with SonarQube


```
.gitignore M X
semester_3 > grcPortal > .gitignore

1 .env
2 instance/
3 __pycache__/_
4 *.pyc
```

Input Validation 🍷

For username and password validation we are using email-id pattern so that the attacker can not enter any random username/script; and for password - it should be minimum 8 characters long (at least 1 alphabet and 1 number)

```
# Validate email
email_pattern = r'^[\w\.-]+@[\w\.-]+\.\w+$'
if not re.match(email_pattern, email):
    error = "Invalid email format!"
    return render_template("register.html", error=error)

# Validate password length and complexity
if len(password) < 8 or not re.search(r'[A-Za-z]', password) or not re.search(r'\d', password):
    error = "Password must be at least 8 characters long and contain at least one letter and one number."
    return render_template("register.html", error=error)
```

Chose 2 OWASP Top 10 Vulnerabilities

1. Weak Password Policy (OWASP A07:2021 – Identification & Authentication Failures)

- **Description:** Weak passwords (like 12345678) could allow brute-force attacks.
- **Steps to Reproduce:**
 1. Navigate to /login.
 2. Enter test@example.com and password 12345678.
 3. Submit registration form.
- **Observed Result:**

GRC Portal

Invalid credentials.

Email

Password

Login

Enforced strong password policy (regex validation, min length, complexity).

2. SQL Injection (OWASP A03:2021 – Injection)

Description: Application may allow malicious input in login form to bypass authentication.

Steps to Reproduce:

1. Navigate to `/login`.
2. Enter `' OR '1'='1` or `' OR '1@gmail.com'='1` in the email field.
3. Enter any password.

Observed Result:

GRC Portal

Email

Password

Login

Don't have an account? [Create new user](#)

⚠ Please include an '@' in the email address. " OR '1'='1' is missing an '@'.

GRC Portal

Email

' OR '1'='1@gmail.com|

Password

.....



A part followed by '@' should not contain the symbol ' '.

Login

Don't have an account? [Create new user](#)

Used parameterized queries (as implemented in `sqlite3` queries).

Secure Coding Standards (Code Examples)

Hash Passwords (Protect User Data)

```
from dotenv import load_dotenv
from flask import Flask, render_template, request, redirect, url_for, session, flash, send_from_directory

from werkzeug.security import generate_password_hash, check_password_hash
from werkzeug.utils import secure_filename as werkzeug_secure
```

```
# Confirm password check
if password != confirm_password:
    error = "Passwords do not match!"
    return render_template("register.html", error=error)

# Hash and save user
hashed_pw = generate_password_hash(password)

conn = sqlite3.connect("instance/app.db")
cur = conn.cursor()
try:
```

Used password hashing so that attacker can't find the original password.

SQL Injection prevention (OWASP A03:2021).

```
conn.exec_driver_sql(
    "INSERT INTO users (email, password_hash, is_verified) VALUES (:e, :p, :v)",
    {"e": "kush786srj@gmail.com", "p": pw, "v": True},
)
```

In the above code “{.....}” allows vulnerability attacks and insert any value which could not be safe for the organization.

The safe way is to use “(....)” as shown below to avoid any vulnerability attack

```
try:
    cur.execute("INSERT INTO users (email, password_hash, is_verified) VALUES (?, ?, ?)",
                (email, hashed_pw, 1))
    conn.commit()
except sqlite3.IntegrityError:
```

Secure error handling

```
if len(password) < 8 or not re.search(r'[A-Za-z]', password) or not re.search(r'\d', password):
    error = "Password must be at least 8 characters long and contain at least one letter and one number."
    return render_template("register.html", error=error)

# Confirm password check
if password != confirm_password:
    error = "Passwords do not match!"
    return render_template("register.html", error=error)

# Hash and save user
hashed_pw = generate_password_hash(password)

conn = sqlite3.connect("instance/app.db")
cur = conn.cursor()
try:
    cur.execute("INSERT INTO users (email, password_hash, is_verified) VALUES (?, ?, ?)",
                (email, hashed_pw, 1))
    conn.commit()
except sqlite3.IntegrityError:
    error = "User with this email already exists!"
    return render_template("register.html", error=error)
finally:

# error handlers
@app.errorhandler(404)
def not_found(e):
    return render_template("errors/404.html"), 404

@app.errorhandler(500)
def server_error(e):
    logging.error(f"500 Error: {e}")
    return render_template("errors/500.html"), 500
```

Handling the errors for safe returning.

